

Phone UI and ESP32 quick start

Two paths are provided: (1) phone connects directly to the ESP32-hosted page; (2) ESP32 uploads/polls a server and the phone opens that server. Both use the same UI and result schema. The demo server uses clearly marked synthetic data and never silently substitutes for hardware.

Quick trial: with Python3.12, open a terminal in software/server and run `python server.py --demo`. Open `http://127.0.0.1:8765` on the computer, enter API key `local-demo-key`, connect, start and stop. Check the DEMO label and four channels. No PCB or paid cloud is needed.

Phone trial: connect phone/computer to trusted Wi-Fi, set different `AEROSENSE_UI_KEY` and `AEROSENSE_DEVICE_KEY` values, then start with `--host0.0.0.0`. Open the computer LAN IP:8765 on the phone. Default binding is loopback. The package does not publish an Internet service or supply cloud credentials.

UI features: bilingual switch, finite block schedule, duty, dark/single/all LED selection, QC, calibration capture, $\Delta F/F_0$ /reference normalization, JSON/CSV export and direct-device raw ADC download. Missing battery/cell-temperature sensors are shown as unavailable.

Flashing and first hardware connection

1. Install PlatformIO. `platformio.ini` pins `espressif32@6.10.0` and `ArduinoJson@6.21.5`. Extract to a short ASCII path, e.g. `C:/AeroSense`, because long Windows paths may prevent compiler subprocesses. Run `pio run` to compile and `pio run -t buildfs` for the UI filesystem.
2. Copy `src/config.example.h` to `src/aerosense_config.h` and set Wi-Fi, AP password and API key. Do not publish the populated file. `BOARD_VALIDATED` defaults false, allowing dark acquisition only. Set true and rebuild after supply, ADC and LED waveform checks.
3. J2 USB-C is not established as a USB-UART bridge. Use a 3.3 V logic USB-UART at J3 with common ground and crossed TX/RX with schematic J3 pin1=3V3_DIG, pin2=TXD0(GPIO1), pin3=RXD0(GPIO3), pin4=GND; verify connector pin1 orientation and leave adapter VCC disconnected when the main supply is on. Never use 5 V UART. Hold GPIO0 low with SW3 and reset with SW2 for download mode; run `pio run -t upload` and `pio run -t uploadfs`, then reset normally.
4. Connect phone to the AeroSense AP, open `http://192.168.4.1` and enter your API key; alternatively use the ESP32 LAN address after joining lab Wi-Fi. AP access is not Internet remote access. Begin with dark blocks and raw ADC checks.

The source was actually compiled. Default firmware contains no personal network credentials and has not been flashed or exercised on the board. Build success verifies compilation/linking/size, not timing under Wi-Fi load.

Communication, retention and cloud

Endpoint	Behavior
GET /api/status	Mode, state, latest block and QC
POST /api/command	start/stop; blocks1-60, intervalSec3-300, duty1-50, ledMask0-15
GET /api/raw.csv	Direct ESP32 latest raw block; rejected while sampling
POST /api/device/result	Device-key upload, ID deduplication, SQLite storage
GET /api/device/poll	Device polls expiring command
POST /api/device/ack	Execution/rejection acknowledgment
POST /api/predict	Research inference; explicit rejection without a model

Use X-Aero-Key for UI and a separate X-Device-Key for the device. Internet deployment requires an HTTPS reverse proxy and a CA certificate on ESP32; no setInsecure is used. ALLOW_LAN_HTTP is for private LAN tests only. Do not expose plaintext control publicly. server.py is a research service requiring deployment, backups and access management for production.

ESP32 holds the latest raw block and up to 8 pending summaries in RAM, stopping new acquisition when the queue fills. Reboot loses unsaved data. Server stores summaries; raw ADC must be downloaded per block or persistent storage added. UI retains the latest 1000 received blocks and does not backfill on reload; use server/api/export.json for full stored history.

Stop is a network command, not a hardwired emergency stop. Each exposure is bounded to 2 s even offline, but an already programmed finite local session can continue. Use SW1 for immediate power removal. UI does not actuate gas valves, temperature or fan speed because those control/sensing connections are absent.

Calibration operation and analysis reproduction

Acquire five dark blocks with LEDs off, adding each completed unique ID to Dark. Use all four LEDs and a matched blank for five Baseline blocks. Apply the reference and capture 5 blocks at predeclared times, adding each to Reference. Clear/repeat calibration after duty, cells or wells change.

Calibration is stored locally in this browser; the key stays in page memory. Exported JSON contains I/Q, amplitudes, calibration records and normalization; CSV is a channel-amplitude table. Keep private experiment records and credentials out of the public wiki package.

software/analysis/dlia.py provides timestamp-aware joint least squares, complex unmixing and 48 synthetic sweeps. Firmware uses fixed-window I/Q; their differences are documented. Run python dlia.py --sweep to regenerate CSV, figure and validation JSON.

NN training CSV columns: sample_id, group, label, D1, D2, D3, D4, with predefined $\Delta F/F_0$ features. Run python odor_model.py train data.csv --holdout batch3 --out trained_model. Require 3 independent groups; holdout does not participate in scaling/tuning. Set AEROSENSE_MODEL to model.json to enable server inference, still labeled research-only.

Quick reproduction: from package root run python reproduce.py --plots to redraw stored numerical results; --optics reruns optics, --flow reruns 27 plate CFD cases plus the no-plate control (NumPy is slower; C# accelerator optional), --video makes MP4. Parameters and seeds are retained.